# Assignment 5 Writeup — Eye Hear U

| Name | Student number |
|---|---|
| Maria Ma | 1009054924 |
| Zhixiao Fu | 1009834342 |
| Siyi Zhu | 1008793076 |
| Chloe Yang | 1009261433 |

---

## Part One: Profiling Execution Time

We profiled **5 key functions** using Python's `cProfile` module. The profiling script lives at `ml/profiling/profile_functions.py` and generates per-function cProfile tables, binary `.prof` files for `pstats` / snakeviz analysis, and a machine-readable JSON summary.

### How to run

```
cd ml/
python -m profiling.profile_functions
```

Binary `.prof` files are saved under `ml/profiling/results/` for deeper analysis.

### Profiled functions (summary)

| # | Function | Location | Wall time | Total calls | Bottleneck |
|---|---|---|---|---|---|
| 1 | `preprocess_video` | `backend/app/services/preprocessing.py` | 0.93 s (3 calls) | 2,726 | `cv2.resize` (77%) |
| 2 | `predict` | `backend/app/services/model_service.py` | 4.11 s (3 calls) | 9,353 | `torch.conv3d` (60%) + `max_pool3d` (30%) |

| | | | | | |
|---|---|---|---|---|---|
| 3 | `i3d_evaluate` | `ml/i3d_msft/evaluate.py` | 6.17 s (1 call, 20 samples) | 15,121 | `torch.conv3d` (96%) |
| 4 | `i3d_train_one_epoch` | `ml/i3d_msft/train.py` | 20.53 s (1 epoch) | 17,895 | `run_backward` (66%) + `conv3d` (30%) |
| 5 | `build_gloss_dict_from_csv` | `ml/i3d_msft/export_label_map.py` | 0.75 s (50 calls × 5K rows) | 2,752,771 | `csv.DictReader.__next__` (56%) |

### Detailed analysis

**1. `preprocess_video` — video preprocessing pipeline**

**What it does:** Decodes an uploaded MP4, applies adaptive frame skip, resizes to short-side-256, pads/trims to 64 frames, center-crops to 224×224, and returns a `(1, 3, 64, 224, 224)` tensor.

**Profile (3 calls on a 90-frame 480×640 video):**

```
ncalls  tottime  cumtime  function
   192    0.716    0.716  cv2.resize              ← 77% of total time
   192    0.050    0.050  cv2.VideoCapture.read    ← 5.4%
     3    0.021    0.021  numpy.asarray            ← 2.3%
   192    0.019    0.019  ndarray.astype           ← 2.0%
```

**Bottleneck:** `cv2.resize` is called **twice per frame** (once for mobile downscale, once for short-side-256 resize), totalling 192 resize calls across 3 videos. Each resize operates on full BGR uint8 frames.

**Improvements we identified:**

1. **Combine the two resize steps into one.** `_load_rgb_frames` first scales 4K frames down to ≤1280 px (mobile guard), then separately scales to short-side-256. These could be computed as a single scale factor, halving `cv2.resize` calls. Estimated savings: ~35% of wall time.
2. **Decode at reduced resolution.** OpenCV decodes full-resolution frames before downscale. Using `ffmpeg` with `-vf scale=...` or hardware-accelerated decoding could decode closer to the target size.
3. **Batch the BGR→RGB conversion.** After stacking into a numpy array, a single `arr[..., ::-1]` slice can reverse channels without a per-frame `cv2.cvtColor` call.

**2. `predict` — model inference (I3D)**

**What it does:** Runs a forward pass through `InceptionI3d`, max-pools over the temporal dimension, applies softmax, and returns top-k predictions.

**Profile (3 calls on a `(1, 3, 64, 224, 224)` tensor, CPU):**

```
ncalls  tottime  cumtime  function
   174    2.456    2.456  torch.conv3d             ← 60% of total time
    39    1.234    1.234  torch.max_pool3d         ← 30%
   213    0.155    0.155  torch._C._nn.pad         ← 3.8%
   171    0.072    0.072  torch.batch_norm          ← 1.8%
```

**Bottleneck:** 3D convolutions dominate (many `conv3d` calls across Inception modules). `max_pool3d` is the second-largest contributor.

**Improvements we identified:**

1. **`torch.jit.trace` or `torch.compile`.** JIT can fuse conv3d + batch_norm + relu sequences and cut kernel-launch overhead; on CPU, `torch.compile(mode="reduce-overhead")` often yields ~10–30% speedup.
2. **Lower input resolution.** For latency-sensitive mobile inference, trying 160×160 or 192×192 could reduce `conv3d` cost while monitoring accuracy.
3. **Half-precision on GPU.** `model.half()` with `torch.cuda.amp` roughly halves conv time on CUDA devices; on CPU, `bfloat16` can help on supported hardware.
4. **ONNX Runtime.** Exporting to ONNX and running with `onnxruntime` (MLAS/oneDNN) can improve CPU inference versus vanilla PyTorch.

### 3. `i3d_evaluate` — I3D evaluation loop

**What it does:** Runs I3D inference over a `DataLoader`, collects predictions, and computes top-k accuracy, MRR, DCG, and a confusion matrix.

**Profile (1 call, 20 samples in 5 batches of 4):**

```
ncalls  tottime  cumtime  function
   100    5.942    5.942  torch.conv3d             ← 96%
   100    0.095    0.095  torch.batch_norm          ← 1.5%
    85    0.028    0.028  torch.relu_               ← 0.5%
```

**Bottleneck:** Almost all time is the model forward pass. Metric computation is negligible.

**Improvements we identified:**

1. **Reuse the inference optimizations above** (JIT, ONNX, resolution, FP16).
2. **Increase batch size** to amortize data transfer and kernel launch (e.g. 16–32 on GPU).
3. **`torch.inference_mode()`** is slightly faster than `@torch.no_grad()` because autograd dispatch is disabled entirely.

### 4. `i3d_train_one_epoch` — single training epoch

**What it does:** Full forward + backward over training batches, loss, and optimizer step.

**Profile (1 epoch, 20 samples in 5 batches of 4, CPU):**

```
ncalls  tottime  cumtime  function
     5   13.586   13.586  run_backward              ← 66%
   100    6.152    6.152  torch.conv3d              ← 30%
   100    0.436    0.436  torch.batch_norm           ← 2.1%
     5    0.000    0.254  optimizer.step             ← 1.2%
```

**Bottleneck:** Backprop ( `run_backward` ) costs about **twice** the forward `conv3d` time because gradients flow through all 3D conv layers and activations are retained.

**Improvements we identified:**

1. **Mixed-precision training (AMP).** `torch.cuda.amp.autocast()` with `GradScaler` on GPU, or `torch.amp.autocast("cpu", dtype=torch.bfloat16)` where supported.
2. **Gradient accumulation** over 2–4 batches to mimic a larger batch without proportional memory growth.
3. **Freeze early backbone layers** during fine-tuning to shrink backward work; our training code already supports freezing via `backbone_freeze_epochs` .
4. **`torch.compile`** on PyTorch 2.x for fused ops and better memory layout (often 10–40% on training loops in favorable setups).

#### 5. `build_gloss_dict_from_csv` — label map construction

**What it does:** Reads a training CSV, extracts unique glosses (lowercased, stripped), sorts them, and returns `{gloss: index}` .

**Profile (50 calls × 5,000-row CSV):**

```
ncalls    tottime  cumtime  function
250050    0.254    0.422   csv.DictReader.__next__  ← 56%
500000    0.051    0.051   dict.get                 ← 6.8%
500000    0.050    0.050   str.strip                ← 6.7%
500002    0.035    0.035   builtins.len             ← 4.7%
250000    0.028    0.028   str.lower                ← 3.7%
```

**Bottleneck:** `csv.DictReader` allocates a new dict per row; 5,000 rows × 50 calls adds up.

**Improvements we identified:**

1. **Use `csv.reader` with a column index** after reading the header once — avoids per-row dict construction. Estimated savings: ~40–50% of this function's time.
2. **Cache** the result with `functools.lru_cache` or a module-level cache when the same CSV is read repeatedly.
3. **`pandas.read_csv`** for very large files can outperform the pure-Python `csv` module.

### Reproducing and exploring profiles

After `python -m profiling.profile_functions` , stdout contains the tables above; `ml/profiling/results/*.prof` and `profile_summary.json` support tooling.

Interactive sorting with `pstats` :

```
python -c "
import pstats
p = pstats.Stats('ml/profiling/results/preprocess_video.prof')
p.sort_stats('cumulative')
p.print_stats(30)
"
```

For flame-style visualization, `snakeviz` can open a `.prof` file:

```
pip install snakeviz
snakeviz ml/profiling/results/predict.prof
```

### Key takeaways

1. **3D convolutions dominate** — `torch.conv3d` accounts for 60–96% of time in model-related paths; this is structural for I3D. The highest-leverage ideas are compile/ONNX, mixed precision, and (where acceptable) smaller spatial input.
2. **Backpropagation is ~2× the forward pass** — in training, `run_backward` is the largest slice; AMP and freezing early layers attack that directly.
3. **Preprocessing is resize-bound** — merging the two resize stages would cut `preprocess_video` time materially (~35% estimated).
4. **CSV parsing overhead is avoidable** — replacing `DictReader` with indexed `csv.reader` removes most dict churn.
5. **Post-processing is negligible** — evaluation metric code is under 1% of `i3d_evaluate`; optimize the forward pass first.

---

## Part Two: Code Coverage

### Coverage tools

- **Backend + ML:** `pytest-cov` (pytest plugin wrapping `coverage.py`), matching the workflow described in the pytest-cov documentation and common open-source Python projects.
- **Mobile:** Jest with Istanbul coverage via the `--coverage` flag (equivalent role for the TypeScript/React Native stack).

### GitHub Actions workflow

The CI workflow ( `.github/workflows/ci.yml` ) runs three parallel jobs on every push and on every pull request targeting **`main`** **or** **`master`** :

| Job | Command | Enforcement | Coverage upload |
|---|---|---|---|
| Backend | `pytest --cov=app --cov-report=xml --cov-fail-under=100` | 100% line and branch on `app/` | Codecov (flag: `backend`) |
| ML | `pytest --cov --cov-report=xml --cov-fail-under=100` | 100% line | Codecov (flag: `ml`) |
| Mobile | `npx jest --coverage --ci` | 100% line + function (`coverageThreshold` in `package.json`) | Codecov (flag: `mobile`, `lcov.info`) |

### README badge

The README includes a live Codecov badge that updates when CI uploads reports:

```
[![codecov](https://codecov.io/gh/MariaMa-GitHub/EyeHearU/branch/main/graph/badge.svg)]
(https://codecov.io/gh/MariaMa-GitHub/EyeHearU)
```

### Codecov PR integration

The `codecov.yml` configures:

- **Project status:** `target: auto` with a **1%** regression threshold
- **Patch status:** new code should reach **80%** coverage
- **Flags:** separate tracking for **`backend`** , **`ml`** , and **`mobile`** , with **carryforward** so partial uploads do not drop the dashboard to zero
- **PR comments:** layout includes reach, diff, flags, and files

### How to generate coverage locally

```
# Backend (100% line + branch)
cd backend
pytest tests/ -v --cov=app --cov-report=term-missing --cov-fail-under=100

# ML (100% line)
cd ml
python -m pytest tests/ -v --cov --cov-report=term-missing --cov-fail-under=100

# Mobile (100% line + function on app/ and services/)
cd mobile
npx jest --coverage
```

---

## Part Three: Test Coverage Breadth

### Current coverage

| Component | Tests | Line coverage | Branch coverage |
|---|---|---|---|
| Backend (`app/`) | 82 pytest | **100%** | **100%** |
| ML (`i3d_msft/`, `modal_train_i3d`) | 191 pytest | **100%** | — |
| Mobile (`app/`, `services/`) | 66 Jest | **100%** | — |
| **Total** | **339 tests** | **100%** on each component's measured tree | |

Mobile Jest also enforces **100% function** coverage on those paths ( `coverageThreshold` in `mobile/package.json` ).

ML line coverage is measured over the packages listed in `ml/.coveragerc` ( `i3d_msft` , `modal_train_i3d` ). Backend line and branch coverage use `backend/.coveragerc` on `app/` .

Coverage is enforced in CI with `--cov-fail-under=100` for backend and ML, and Jest `coverageThreshold` for mobile. Any change that drops below those targets fails CI.

### What is covered

**Backend (82 tests):**

- API endpoints: `/health` , `/ready` , `POST /api/v1/predict`
- Video preprocessing: frame decode, resize, crop, normalize, pad, temporal sampling
- Model service: S3 download, label map loading, inference, top-k predictions
- Lifespan: model load success/failure, startup error handling
- Firebase: mocked Firestore integration

**ML (191 tests):**

- I3D backbone: all Inception modules, forward pass, weight loading, logits replacement
- I3D dataset: frame loading, frameskip, normalization, padding, tensor conversion
- I3D training: `train_one_epoch` , evaluate, checkpoint handling, S3 upload, optimizer building
- I3D evaluation: top-k hits, MRR, DCG, split parsing, device detection
- S3 data: client setup, split downloads, clip subset management
- Export/build label map: CSV parsing, JSON writing, S3 upload
- Video transforms: random crop, center crop, horizontal flip
- Modal wrapper: run name parsing, command building, checkpoint resolution

**Mobile (66 tests):**

- API service: health check, predict, URL resolution (including LocalTunnel `loca.lt` header), error explanations

- Home and root layout: landing screen, stack layout, navigation to camera and history
- Camera screen: permissions, recording, countdown completion, upload, camera toggle, prediction display, TTS
- History screen: empty state, rendering, time formatting, clear history, storage errors

---

## Part Four: Test Coverage Depth

### Chosen module: `backend/app/services/preprocessing.py`

**Why this module?** `preprocessing.py` is the critical accuracy path between a raw phone video and the I3D model input tensor. Every frame passes through temporal sampling, spatial resize, normalization, padding, and center-cropping. A bug in any stage silently degrades prediction accuracy — the root cause of our original low-accuracy bug was a spatial resize error here. The module must handle arbitrary aspect ratios (portrait 9:16, landscape 16:9, square, ultra-wide), extreme resolutions (4K, tiny webcam), variable-length videos, corrupt inputs, and missing dependencies.

### Test file: `backend/tests/test_preprocessing_depth.py`

The file defines `TestPositive` and `TestNegative` classes with **16 targeted tests** (10 positive, 6 negative). The **module-level docstring** states why `preprocessing.py` was chosen and what failure families we guard against; **each test method** has a docstring that names the edge case or failure mode, ties it to model or user impact, and states what the assertions prove. The tables below mirror that mapping for readers who start from this writeup.

**Positive tests (correct behavior)**

| # | Test | Edge case / use case | Why it matters |
|---|---|---|---|
| 1 | `test_portrait_9_16_preserves_spatial_detail` | Portrait phone video (1080×1920) | **The original accuracy bug** — old pipeline crushed portrait width to ~144 px |
| 2 | `test_4k_video_downscaled_before_resize` | 4K video (3840×2160) | Without mobile cap, 4K frames waste memory and amplify codec artifacts |
| 3 | `test_single_frame_video_padded_to_64` | Video with only 1 frame | Very short recordings must pad to 64 frames, not crash |
| 4 | `test_normalization_range_minus1_to_plus1` | [−1, 1] normalization | I3D uses [−1, 1], not ImageNet mean/std — wrong normalization silently degrades accuracy |
| 5 | `test_full_pipeline_output_shape_and_dtype` | End-to-end pipeline | Model expects exactly (1, 3, 64, 224, 224) float32 — any deviation crashes or mismatches |
| 6 | `test_frameskip_adapts_to_high_fps_video` | 200-frame 60 fps video | Without adaptive frameskip, high-fps video oversamples the start and misses the end |
| 7 | `test_square_video_no_aspect_distortion` | Square 1:1 video | Short-side-256 must work when both sides are equal |
| 8 | `test_center_crop_extracts_center_region` | Crop geometry | ASL signs are centered — off-center crop cuts off hands/fingers |
| 9 | `test_resize_uses_area_interpolation_when_shrinking` | Downscale interpolation | `INTER_AREA` avoids aliasing; `INTER_LINEAR` can add artifacts when shrinking |

| | | | |
|---|---|---|---|
| 10 | `test_resize_uses_linear_interpolation_when_enlarging` | Upscale interpolation | `INTER_AREA` is undefined for upscaling; `INTER_LINEAR` gives smooth results |

**Negative tests (error handling)**

| # | Test | Failure mode | Why it matters |
|---|---|---|---|
| 11 | `test_zero_frame_video_raises_value_error` | Corrupt container, 0 frames | Must raise `ValueError`, not pass an empty tensor to the model |
| 12 | `test_all_reads_fail_raises_value_error` | Valid header, corrupt frame data | Truncated files must not produce garbage predictions |
| 13 | `test_center_crop_rejects_undersized_frames` | Frames smaller than crop size | Defensive check against upstream resize failure |
| 14 | `test_missing_opencv_raises_runtime_error` | `cv2` not installed | Clear error instead of a cryptic `ImportError` |
| 15 | `test_temp_file_cleanup_failure_does_not_crash` | `os.unlink` fails (permissions) | Cleanup `OSError` must not invalidate an otherwise valid prediction |
| 16 | `test_decode_error_propagates_through_pipeline` | Unexpected codec failure | Unexpected errors propagate to the caller, not swallowed |

---

## Part Five: Two Memorable Bugs

We recorded **video walkthroughs** for two bugs. The same recordings are available from **OneDrive** and **University of Toronto Libraries Media** so a link still works if one host is down.

- **Folder (OneDrive):** https://utoronto-my.sharepoint.com/:f:/g/personal/zx_fu_mail_utoronto_ca/IgBR75GGL4K8QKH950mXqxCyAavsmy9elaImVMED8Aiv3DM?e=sFqudf
- **Walkthrough 1 (U of T Libraries):** https://play.library.utoronto.ca/watch/77792bbc37a4bef95fcc2b50d6c596a7
- **Walkthrough 2 (U of T Libraries):** https://play.library.utoronto.ca/watch/45e56ad07a3300d11cc4e79dc7bcc396